Ryan Allen
Gavin Tank

CIS-421 Project Report

Pharmacy Database

Source Code: https://github.com/ryanaiien/cis-421-project

1. Application Background

PharmDB is a relational database for a regional group of retail pharmacies operating across Michigan, with locations in Dearborn, Detroit, Ann Arbor, Ypsilanti, and Lansing. Each store employs pharmacists and technicians, stocks medication from multiple manufacturers, serves local patients, and fills prescriptions written by outside doctors.

This is a strong fit for a relational database because the business depends on clear relationships and consistent records across many connected entities. A prescription links a doctor, patient, pharmacist, and drug. Each pharmacy maintains its own inventory and pricing, patients may have multiple phone numbers, and pharmacies contract directly with manufacturers. A normalized design keeps the data accurate while still supporting useful reporting and day-to-day operations.

In addition to the database itself, the project includes a Django admin UI that connects directly to the same SQLite database for browsing and maintaining live records.

Ryan Allen
Gavin Tank

2. Database Requirements

The database captures the main people, products, and transactions involved in a regional retail pharmacy operation.

- **Pharmacy** stores a name, address, and phone number. Each pharmacy employs staff, sells drugs, and signs manufacturer contracts.
- **Employee** stores personal and scheduling information and specializes into either pharmacist or pharmacy technician.
    - **Pharmacist** stores license and degree information and may be assigned as a patient's primary pharmacist or recorded as the dispensing pharmacist on a prescription.
    - **Pharmacy Technician** stores certification information and supports pharmacists with inventory, prescription preparation, and routine pharmacy operations.
- **Doctor** stores contact and specialty information and writes prescriptions.
- **Patient** stores demographic, insurance, and address information, may have multiple phone numbers, and is assigned a primary pharmacist.
- **Drug** stores a trade name, manufacturer, stock quantity, and controlled substance status.
- **Drug Manufacturer** stores company and address information and produces one or more drugs.
- **Prescription** records the doctor who prescribed the drug, the pharmacist who dispensed it, the patient who received it, the drug, the date, and the quantity.
- **Contract** tracks business agreements between pharmacies and manufacturers.
- **Pricing** tracks the selling price of each drug at each pharmacy.

Several design decisions are especially important:

- The employee specialization is implemented as a disjoint IS-A hierarchy: every employee is either a pharmacist or a technician, but not both.
- Patient phone numbers are modeled as a separate relation because `phone` is a multi-valued attribute.
- Age is treated as a derived attribute and computed from `birthday` when needed rather than stored.
- `Prescription` includes both `doctor_id` and `pharmacist_id` because writing a prescription and dispensing it are different roles in the business process.

Ryan Allen
Gavin Tank

3. ER Diagram

- The ER diagram for PharmDB centers on the main operational entities: `Pharmacy`, `Employee`, `Doctor`, `Patient`, `Drug`, `DrugManufacturer`, and `Prescription`, along with the supporting relationships for pricing, patient phone numbers, and manufacturer contracts.

The ER model highlights several core design choices:

- `Employee` specializes into `Pharmacist` and `PharmacyTechnician` using a disjoint IS-A hierarchy.
- `Patient` includes a multi-valued phone attribute, which becomes the `PatientPhone` relation in the logical schema.
- Age is a derived attribute based on `birthday`, so it is shown conceptually but not stored physically.
- `Prescription` serves as the central transaction entity tying together doctor, pharmacist, patient, and drug.
- `PharmacySells` and `PharmacyContract` resolve the two many-to-many business relationships in the model.

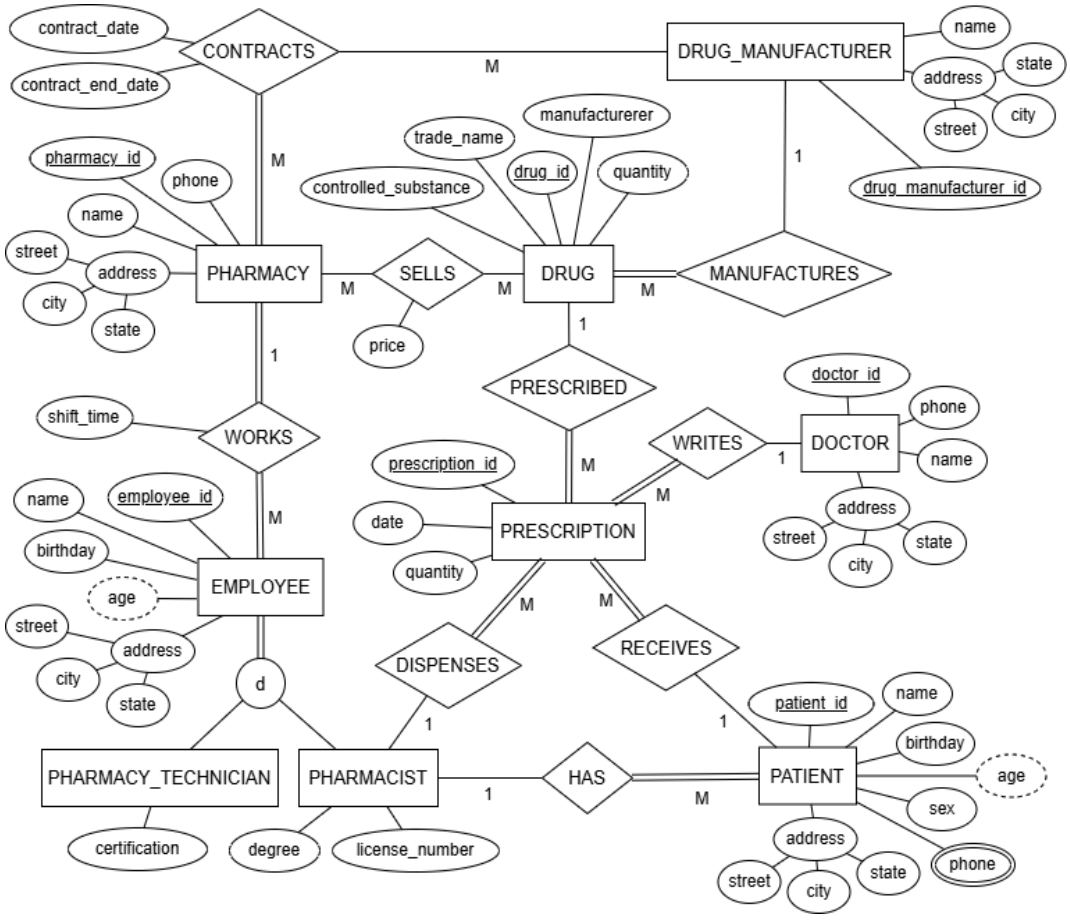

Figure 3.1. PharmDB Entity Relationship Diagram

4. Relational Schema

The logical schema derived from the ER model is shown below. Primary keys are underlined, and foreign keys are noted inline.

- `Pharmacy`(pharmacy_id, name, street, city, state, phone)
- `Employee`(employee_id, pharmacy_id (FK -> `Pharmacy.pharmacy_id`), name, street, city, state, birthday, employee_type, shift_time)
- `Pharmacist`(employee_id (FK -> `Employee.employee_id`), degree, license_number)
- `PharmacyTechnician`(employee_id (FK -> `Employee.employee_id`), certification)
- `Doctor`(doctor_id, name, specialty, phone, street, city, state)
- `Patient`(patient_id, name, sex, insurance, birthday, street, city, state, primary_pharmacist_id (FK -> `Pharmacist.employee_id`))
- `PatientPhone`(patient_id, phone_number, patient_id (FK -> `Patient.patient_id`))
- `Drug`(drug_id, trade_name, manufacturer_id (FK -> `DrugManufacturer.manufacturer_id`), quantity, controlled_substance)
- `DrugManufacturer`(manufacturer_id, name, street, city, state)
- `PharmacySells`(pharmacy_id, drug_id, pharmacy_id (FK -> `Pharmacy.pharmacy_id`), drug_id (FK -> `Drug.drug_id`), price)
- Prescription(prescription_id, doctor_id (FK -> Doctor.doctor_id), pharmacist_id (FK -> `Pharmacist.employee_id`), patient_id (FK -> Patient.patient_id), drug_id (FK -> Drug.drug_id), prescription_date, quantity)
- PharmacyContract(pharmacy_id, manufacturer_id, pharmacy_id (FK -> Pharmacy.pharmacy_id), manufacturer_id (FK -> `DrugManufacturer.manufacturer_id`), contract_date, contract_end_date)

This schema stays close to the conceptual model while keeping the design clean and normalized. The IS-A hierarchy is implemented with a discriminator column in Employee plus subtype tables for role-specific attributes. The multi-valued patient phone attribute becomes its own table, and age is treated as a derived value rather than a stored attribute. Full DDL is documented in `schema/relational-schema.md`.

Ryan Allen
Gavin Tank

5. Sample Database Instance

The sample database is large enough to support meaningful joins, aggregates, and updates without becoming difficult to follow. It includes five stores, eight doctors, twenty patients, thirty drugs, and forty prescriptions spread across the participating pharmacies.

Table	Rows
`Pharmacy`	5
`Employee`	15
`Pharmacist`	6
`PharmacyTechnician`	9
`Doctor`	8
`Patient`	20
`PatientPhone`	26
`Drug`	30
`DrugManufacturer`	8
`PharmacySells`	50
`Prescription`	40
`PharmacyContract`	19

Representative rows from several key tables are shown below to illustrate the kind of data stored in the system.

Patient

patient_id	name	insurance	city	primary_pharmacist_id
1	John Doe	BlueCross	Dearborn	1
2	Jane Roe	Aetna	Detroit	3
3	Mike Lee	Cigna	Ann Arbor	5
4	Sara Kim	UnitedHealth	Dearborn	1

Ryan Allen
Gavin Tank

patient_id	name	insurance	city	primary_pharmacist_id
5	Tom Clark	Humana	Detroit	3

Drug

drug_id	trade_name	manufacturer_id	quantity	controlled_substance
1	Aspirin	1	100	0
2	Ibuprofen	2	200	0
3	Amoxicillin	3	150	0
4	Metformin	1	120	0
5	Lisinopril	2	180	0
11	Gabapentin	1	110	1

Prescription

prescription_id	patient_id	doctor_id	pharmacist_id	drug_id	prescription_date	qty.
1	1	1	1	1	2026-01-10	30
2	4	2	1	4	2026-01-12	90
3	9	1	1	6	2026-01-14	30
4	14	4	1	18	2026-01-16	60
5	1	2	1	2	2026-01-18	20

The full insert set is documented in `schema/sql-statements.md`.

Ryan Allen
Gavin Tank

6. SQL Statements and Query Results

All SQL statements are collected in `queries/queries.md`, and the full outputs are captured in `queries/query-outputs.md`. Together, the queries demonstrate filtering, aggregation, joins across multiple relationships, relational division, and operational updates.

1. **Patients with prescriptions from more than one doctor.** This query helps identify patients with more complex care histories. Result: 15 patients had prescriptions from at least two distinct doctors.
2. **Most commonly prescribed drug per doctor specialty.** This query compares prescribing patterns across specialties. Result: most specialties produced ties at one prescription each, while Gastroenterology had a clear top result with `Carvedilol` at 2 prescriptions.
3. **Pharmacies that sell every drug made by Pfizer.** This is the project's relational division query. Result: `HealthPlus Pharmacy` was the only store that stocked every Pfizer drug in the sample data.
4. **Revenue per pharmacy.** This query compares store performance by joining prescription quantities to store-specific prices. Result: `MediTrust Pharmacy` ranked first at `7750.65`, followed by `CareWell Pharmacy` at `7455.45`.
5. **Patients with multiple phone numbers.** This query confirms the usefulness of the multi-valued patient phone design. Result: 6 patients had two phone numbers on file.
6. **Doctors who have prescribed at 3 or more pharmacies.** This query finds doctors whose prescriptions are filled across a broader service area. Result: 5 doctors met the threshold, each with prescriptions filled at 3 distinct pharmacies.
7. **Drugs stocked at exactly one pharmacy.** This query highlights exclusive or low-distribution inventory. Result: 11 drugs were stocked by exactly one pharmacy.
8. **Average drug price comparison across pharmacies.** This query compares price spread for drugs sold by multiple stores. Result: 19 drugs were sold by more than one pharmacy; for example, `Aspirin` ranged from `5.49` to `6.49` with an average price of `5.99`.
9. **Pharmacists and their patient counts, ranked.** This query summarizes dispensing workload by pharmacist and store. Result: `Emma Davis` led with 6 unique patients, followed by `Alice Smith` and `Carol White` with 5 each.
10. **Prescriptions filled in the last 30 days.** This query supports recent operational reporting based on the latest prescription date in the database. Result: it returned prescriptions from `2026-01-27` through `2026-02-25`, with the newest record showing `Ivan Cruz` receiving `Aspirin` dispensed by `Emma Davis`.
11. **Patients whose primary pharmacist has never dispensed to them.** This query identifies gaps between assigned care relationships and actual dispensing history. Result: `Fiona Grant` was the only patient matching this condition.
12. **Update drug prices by manufacturer.** This update applies a bulk pricing change based on supplier. Result: every AstraZeneca drug carried by a store had its price increased by 10 percent.

13. **Transfer an employee to a different pharmacy.** This update demonstrates a targeted operational change. Result: `Laura Martinez` was moved to `MediTrust Pharmacy` and her shift was changed to `Morning`.
14. **Pharmacists licensed to dispense controlled substances.** This query links pharmacist credentials to controlled-drug dispensing activity. Result: 4 pharmacists appeared in the result set, covering `Gabapentin`, `Tramadol`, and `Trazodone`.

For the two update queries, the affected data before and after each change is shown below.

Query 12: AstraZeneca price update

trade_name	pharmacy_id	price_before	price_after
Metoprolol	1	12.49	13.74
Omeprazole	2	15.99	17.59
Albuterol	3	21.99	24.19
Rosuvastatin	3	31.49	34.64
Omeprazole	4	14.99	16.49
Albuterol	4	20.99	23.09
Metoprolol	4	11.49	12.64

Query 13: Employee transfer

employee	pharmacy_before	shift_before	pharmacy_after	shift_after
Laura Martinez	CareWell Pharmacy	Evening	MediTrust Pharmacy	Morning

Ryan Allen
Gavin Tank

7. User Interface

In addition to the database schema and SQL queries, the project includes a web interface built with Django Admin. The web app connects directly to the same `PharmacyDB.db` file used for the SQL portion of the project, so the interface works with the live database rather than a copied dataset. This made it easy to demonstrate that the project supports not only querying and updates in SQL, but also practical record browsing and data entry through an administrative interface.

The Django admin UI is especially useful for this project because the database contains many linked entities. It provides a simple way to inspect records, follow foreign-key relationships, and create or edit entries without writing SQL by hand. In the screenshots below, the interface shows that the schema maps cleanly into a working application with list views, create forms, and success feedback after updates.

7.1 Database Admin Homepage

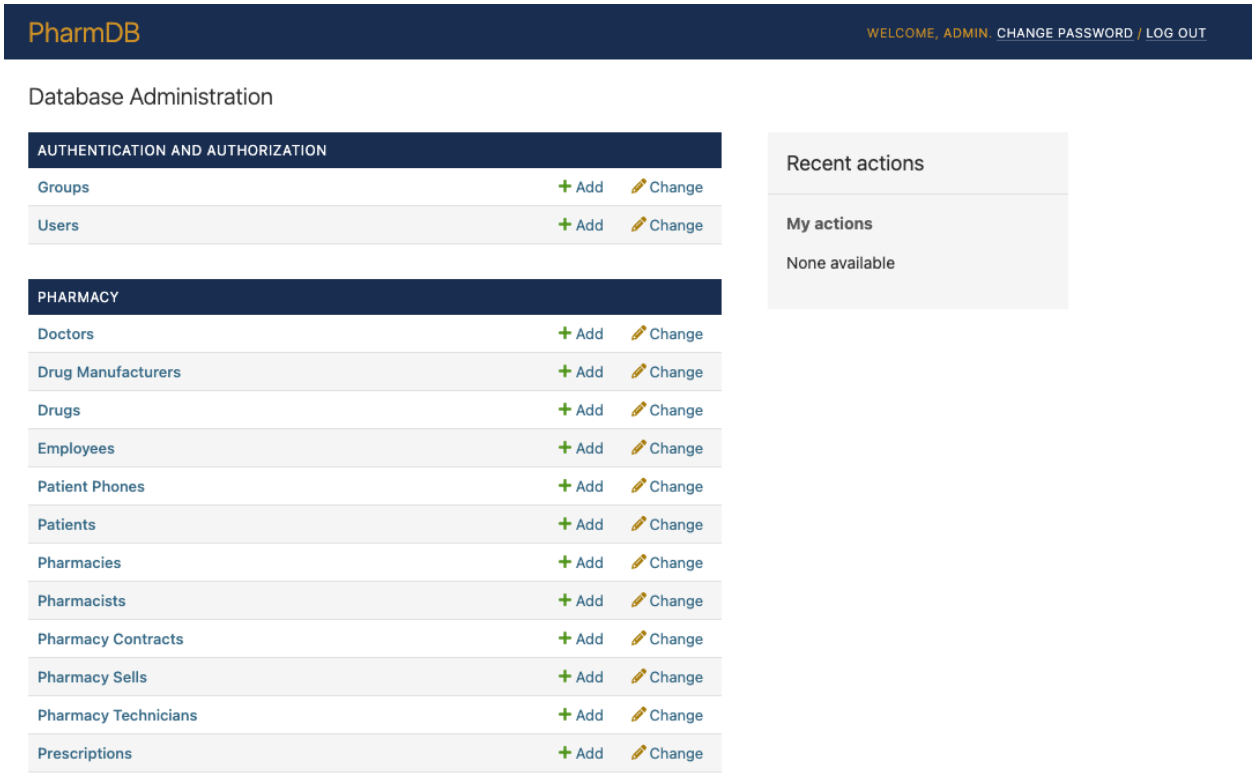

Figure 7.1. Database Admin Homepage. The admin homepage serves as the main entry point into the application and lists the database tables exposed through Django Admin. From this screen, an administrator can quickly navigate to entities. This view shows that the schema supports a practical management interface, not just SQL scripts.

Ryan Allen
Gavin Tank

7.2 Database Admin Prescriptions Page (List View)

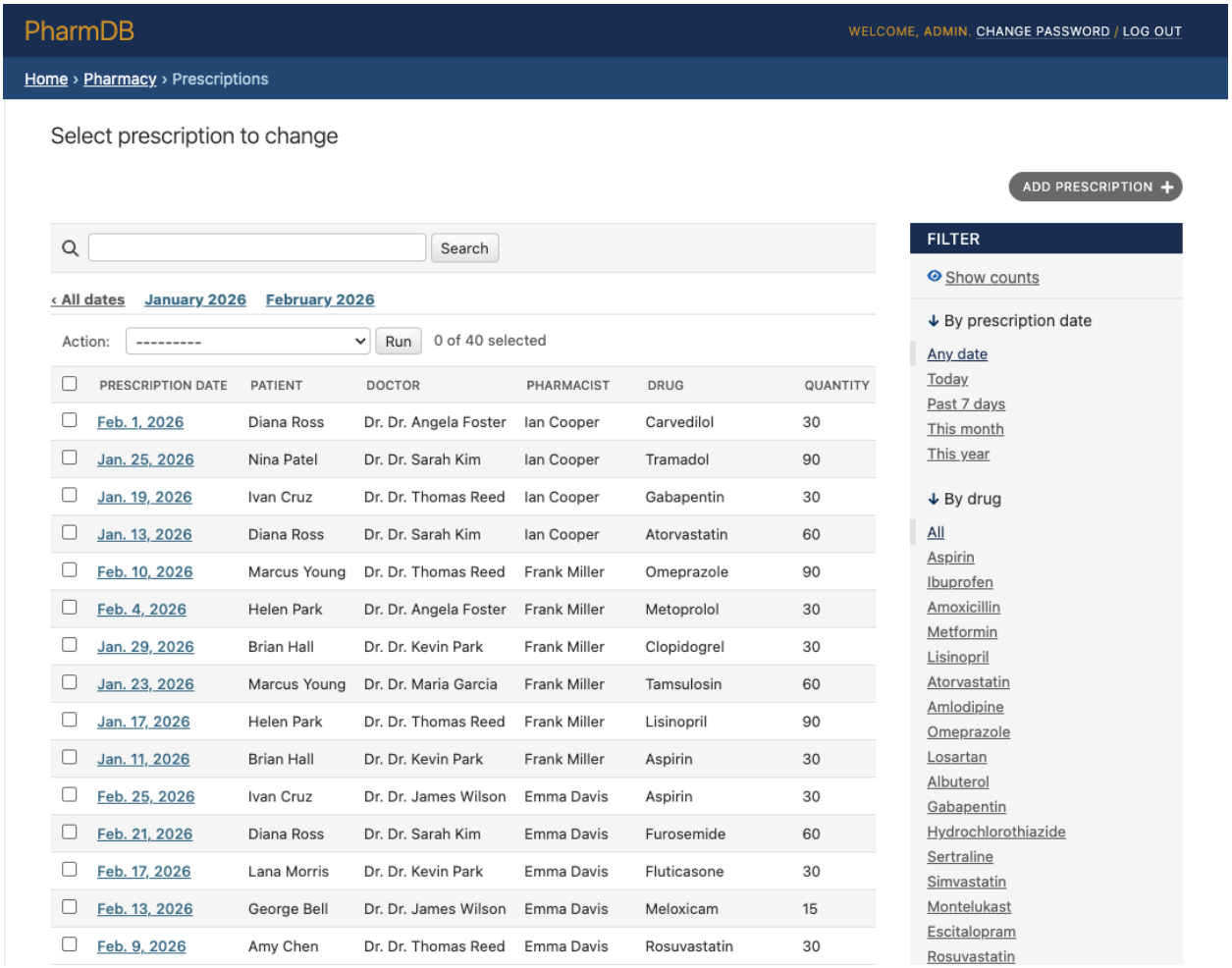

Figure 7.2. Database Admin Prescriptions Page (List View). The prescriptions list page displays prescription records in a searchable, browsable table. It lets an administrator review multiple prescriptions at once and quickly see the core business relationship captured by the database: doctor, patient, pharmacist, drug, date, and quantity. This view shows how the underlying foreign-key relationships become readable in the interface.

Ryan Allen
Gavin Tank

7.3 Database Admin Add Prescription Page (Create View)

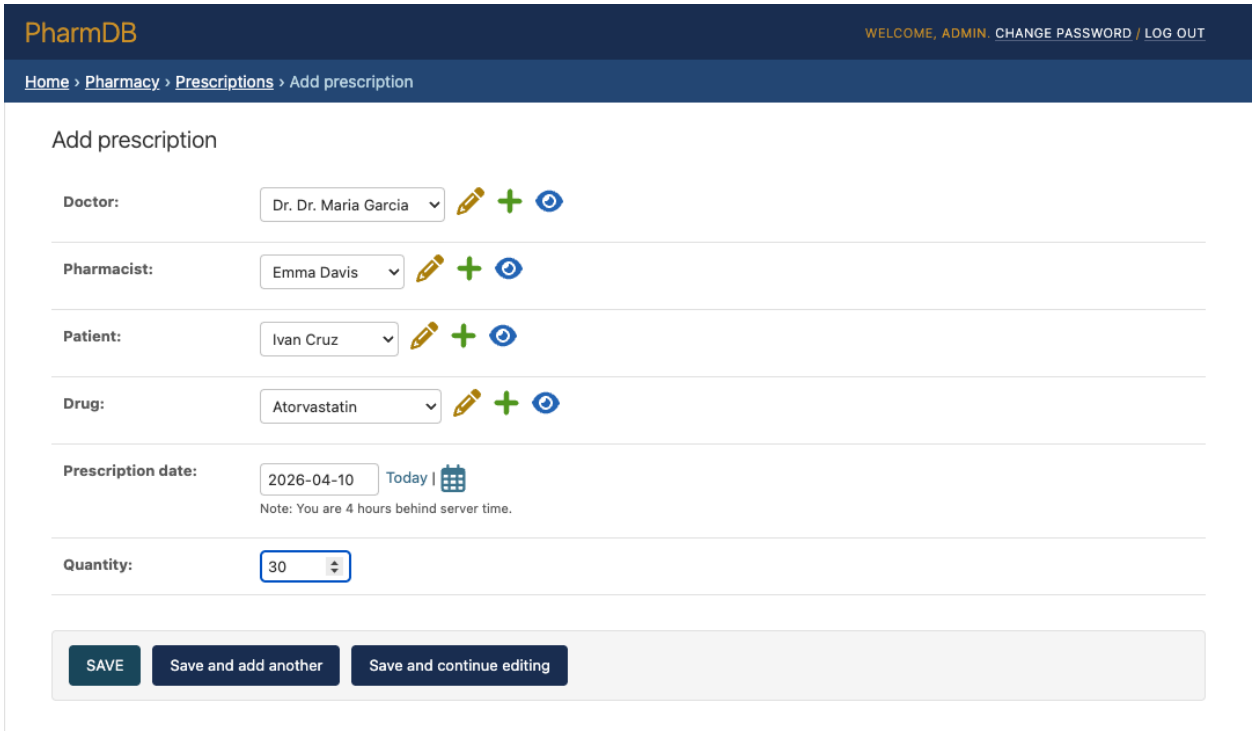

Figure 7.3. Database Admin Add Prescription Page (Create View). The add prescription form allows an administrator to create a new prescription by selecting existing related records and entering the prescription date and quantity. This view demonstrates structured data entry on top of the relational schema, with form fields corresponding directly to the attributes and foreign keys defined in the database.

7.4 Database Admin Save Confirmation

Figure 7.4. Database Admin Save Confirmation. After the administrator clicks the *"SAVE"* button on the add prescription form, the application redirects back to the prescriptions list view and displays a success notification confirming that the new record was saved. This shows that the web app is fully connected to the database and supports a complete create workflow from data entry to visual confirmation.

Ryan Allen
Gavin Tank

8. Team Responsibilities

The project was completed collaboratively, with each team member taking primary ownership of specific design, implementation, and documentation work.

Member	**Responsibilities**
Ryan Allen	Refined the entity design, expanded the sample dataset, developed queries 10-14, improved several query cases, built the Django admin interface, and prepared the written report.
Gavin Tank	Designed the ER diagram, finalized the relational schema and DDL, prepared the insert statements and database setup, developed queries 1-9, captured query outputs, and prepared the presentation materials.